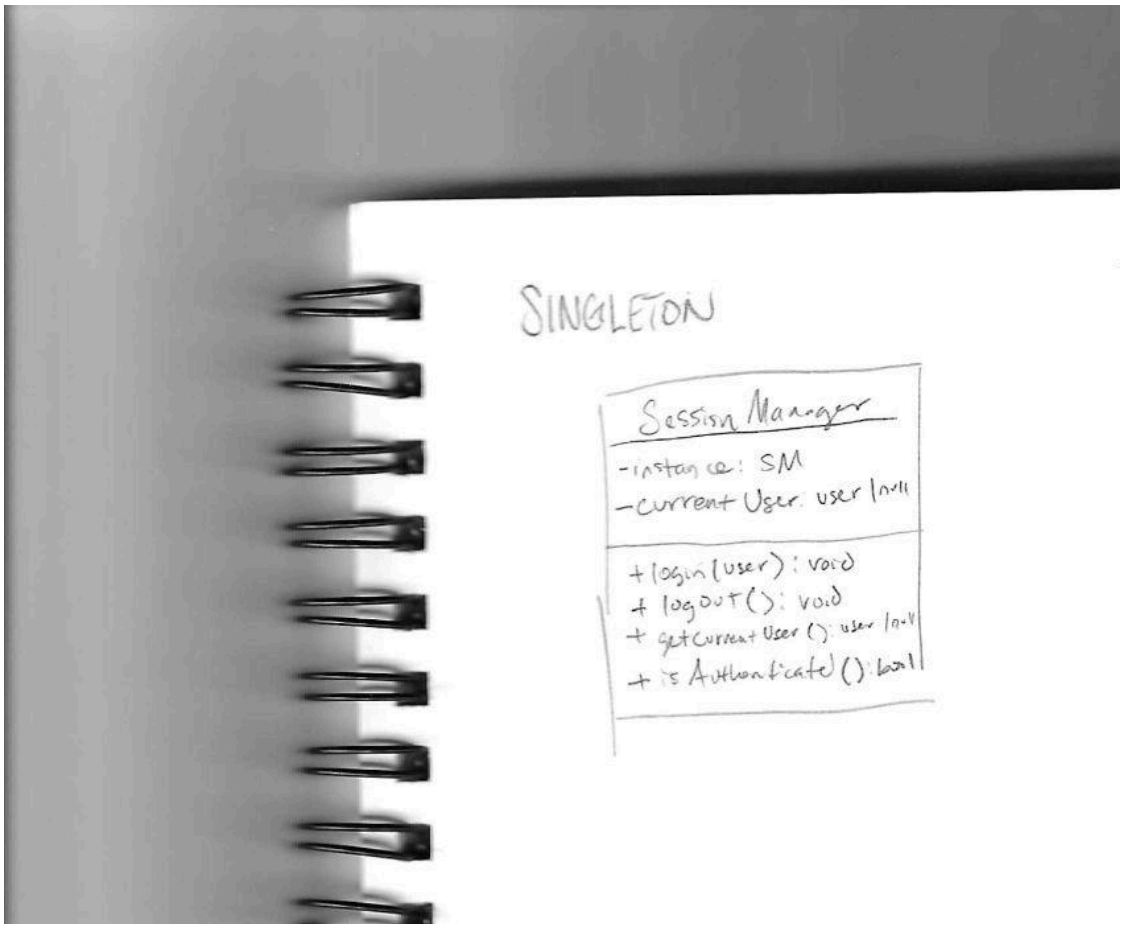
Term Project

CIS-476

Kyle Breen-Bondie

1) **SINGLETON Pattern**

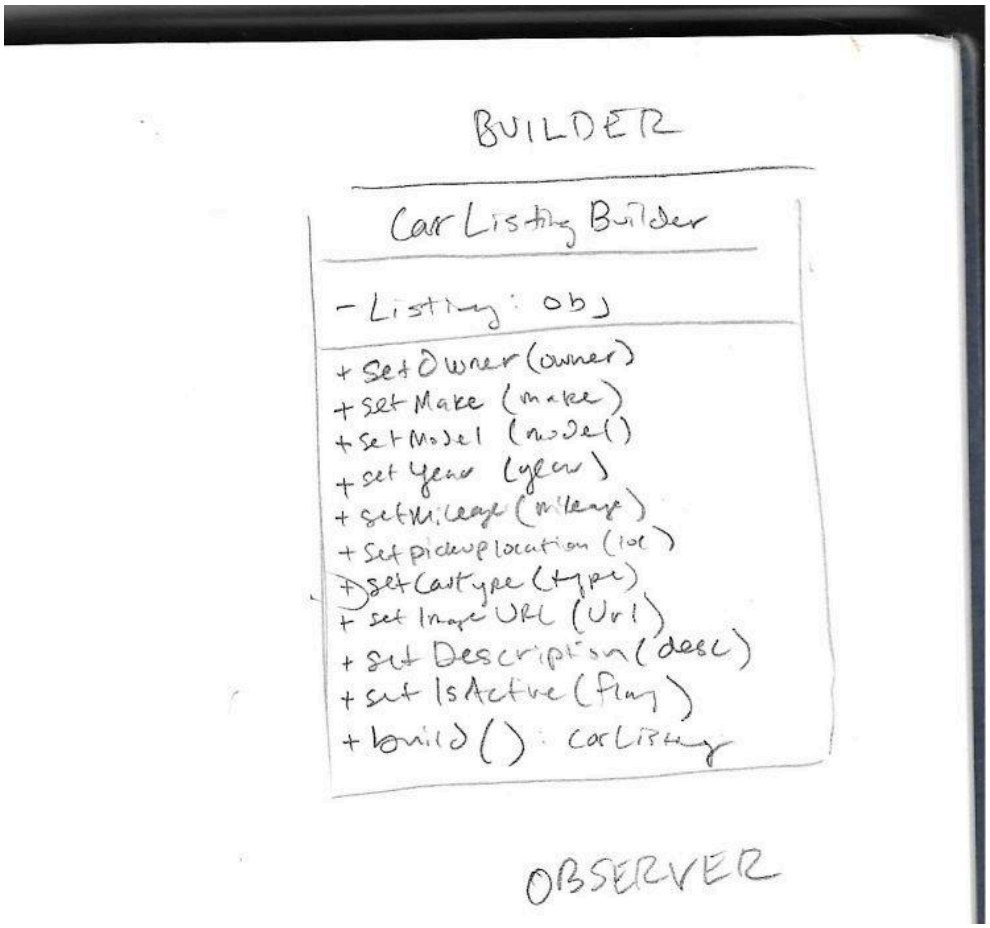
The Singleton manages the currently logged in user in one centralized place. With the current implementation, only one user can be logged in at a time via the Singleton setup, so the authentication state is controlled consistently. It's all handled by the SessionManager.js class.



2) **BUILDER Pattern**

The builder pattern is used to construct car listing objects and helps keep things organized, especially since some of the fields are optional.This is all handled by the CarListingBuilder.js class.

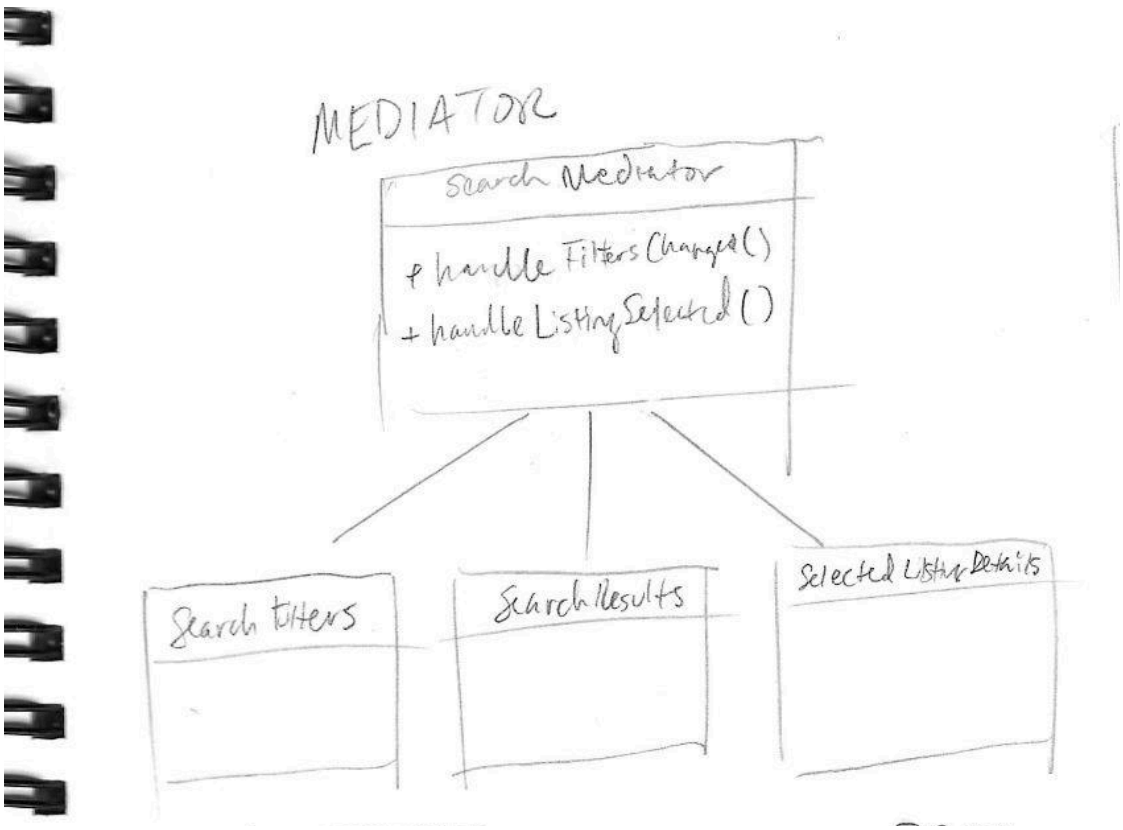
When an owner user-type creates a listing, the backend route makes the class to setOwner(owner), setMake(make), etc.



3) **MEDIATOR Pattern**

The mediator pattern is used on the search page so the UI components do not control one another. Instead of having the search filter directly update the results, a mediator handles those interactions.

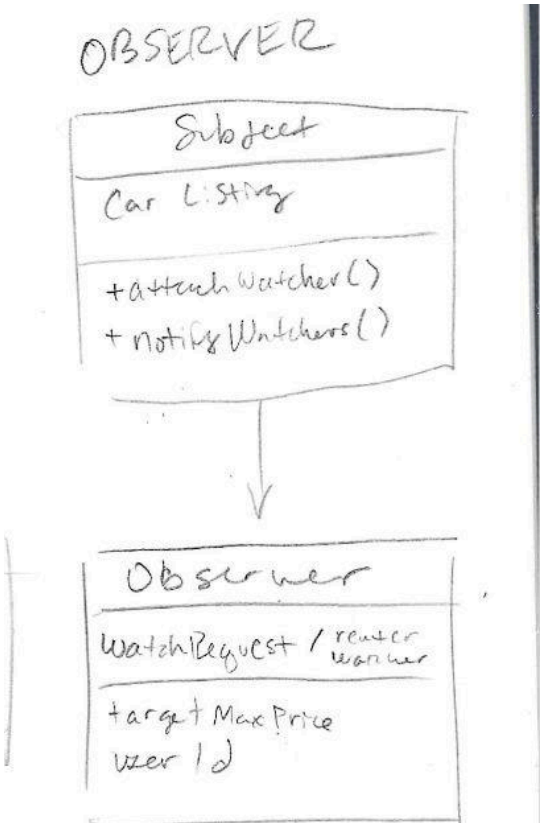
Several files use this, most obviously [SearchMediator.js](), SearchFilters.jsx, SearchResults.jsx, SelectedListingDetails.jsx, and the SearchPage.jsx.



4) **OBSERVER Pattern**

The observer pattern is used in the watchlist notification system. After an owner lowers the price on a listed car, the system checks all watchers and notifies if any trigger the condition.

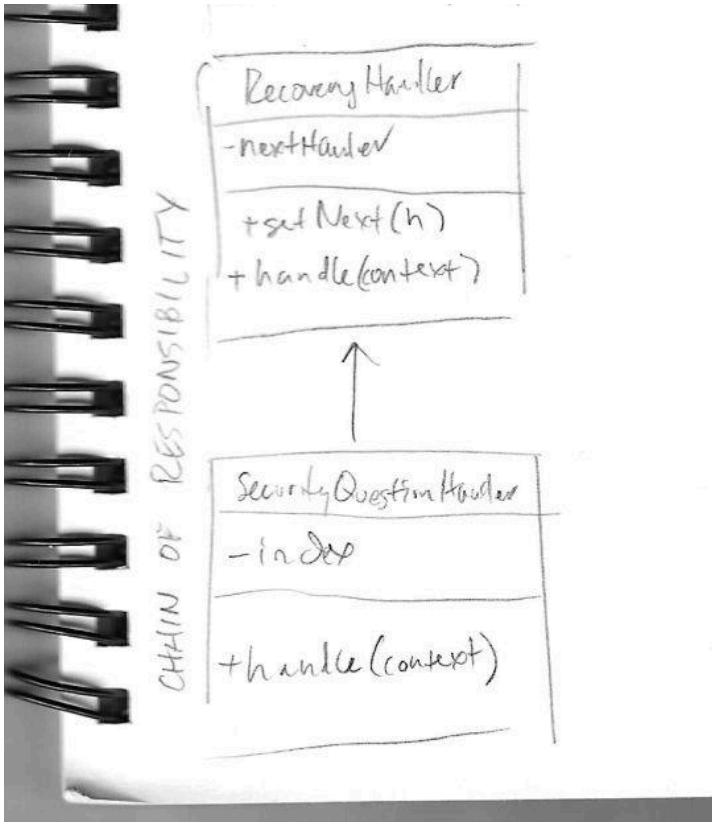
This is implemented inside the server.js file. The observer is a renter watch request and works with attach and notify operations.



5) **CHAIN OF RESPONSIBILITY Pattern**

This pattern is used for password recovery by using the three required security questions. Each question is treated as a handler in a chain. The user must pass each one in sequence. This is used in the RecoveryHandler.js, SecurityQuestionHandler.js, passwordRecoveryService.js.

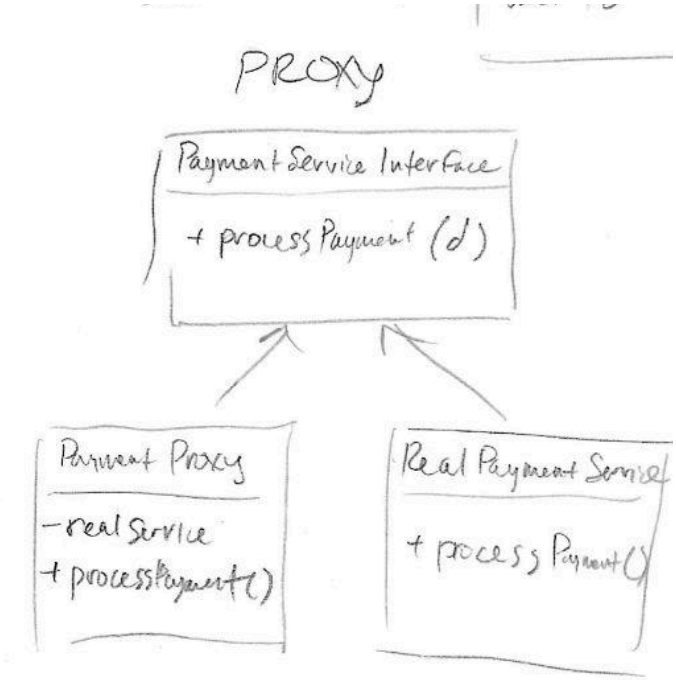
The service creates three SecurityQuestionHandler objects. Each handler checks its own answer and then passes to the next. If any fail, recovery stops, otherwise recovery is successful.



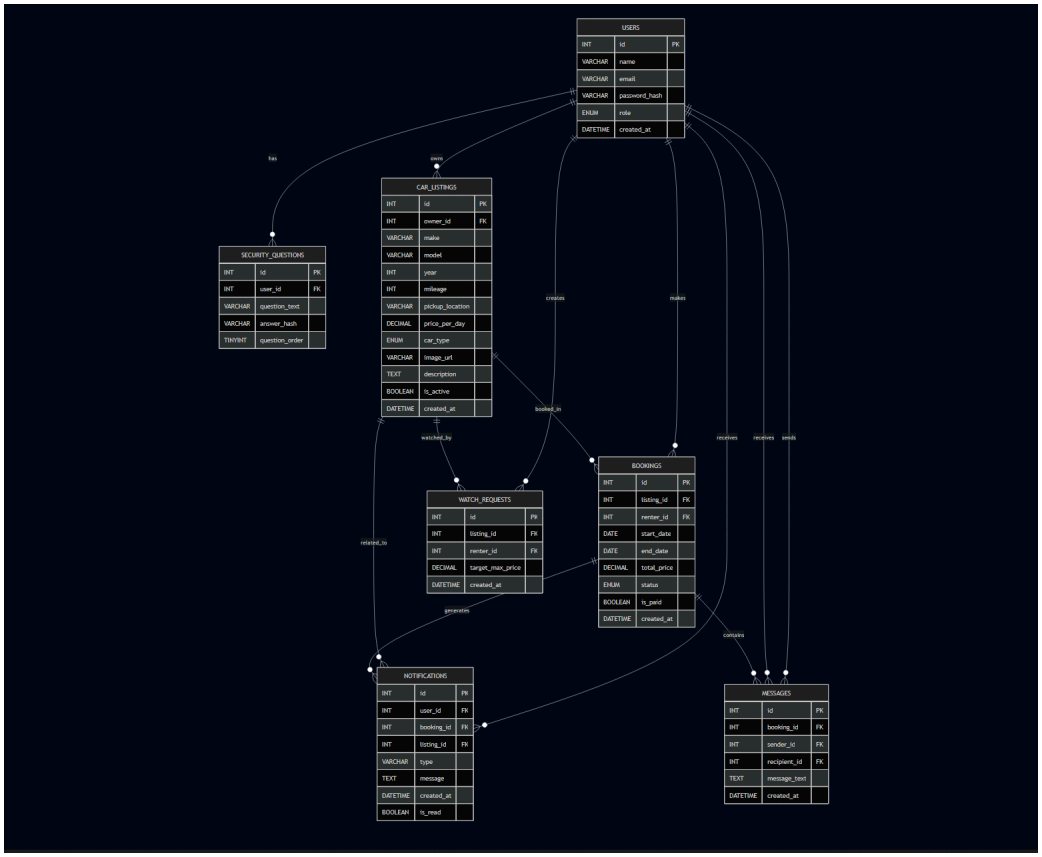
6) **PROXY Pattern**

The proxy pattern is used for the simulated payment feature. Instead of booking and payment logic directly to the payment system, a proxy stands in front and performs checks before allowing payment through. This is done through [PaymenyServiceInterface.js](), [PaymentProxy.js](), [RealPaymentService.js](), [paymentAppService.js]().

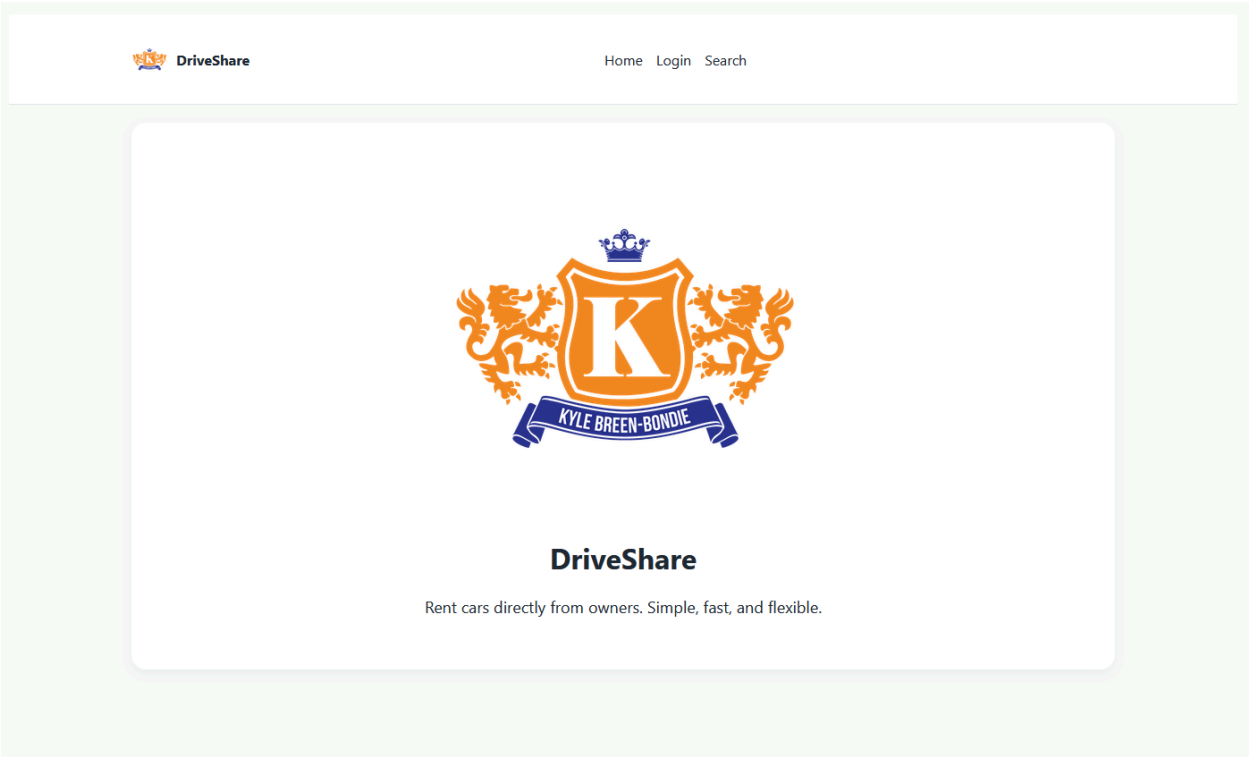
The renter clicks pay and payment logic goes through the proxy, which validates and forwards it. Then payment is marked complete and notifications are created.



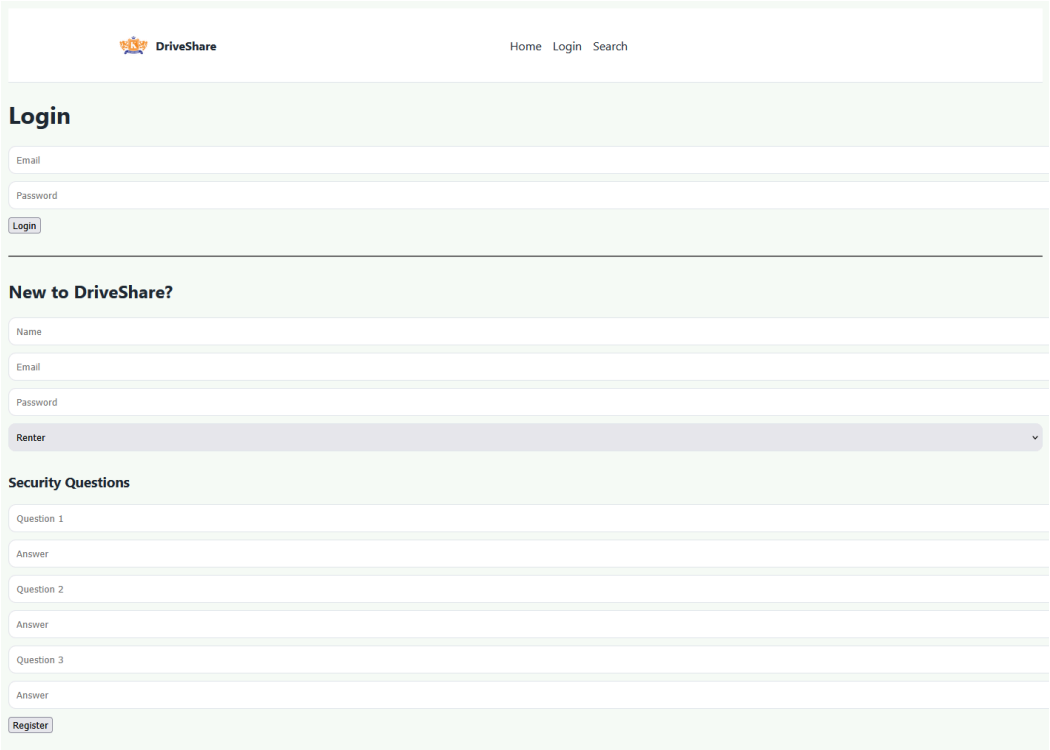
DATABASE Schema



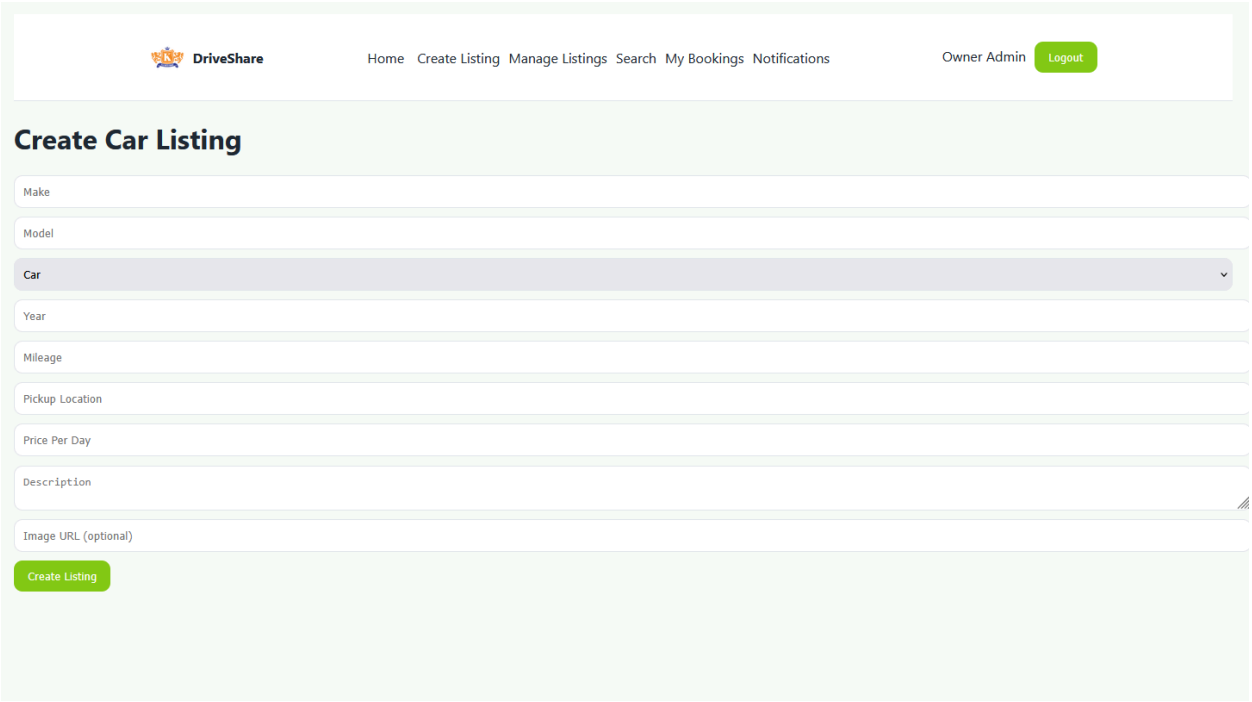
Home / Landing Page (not logged in)



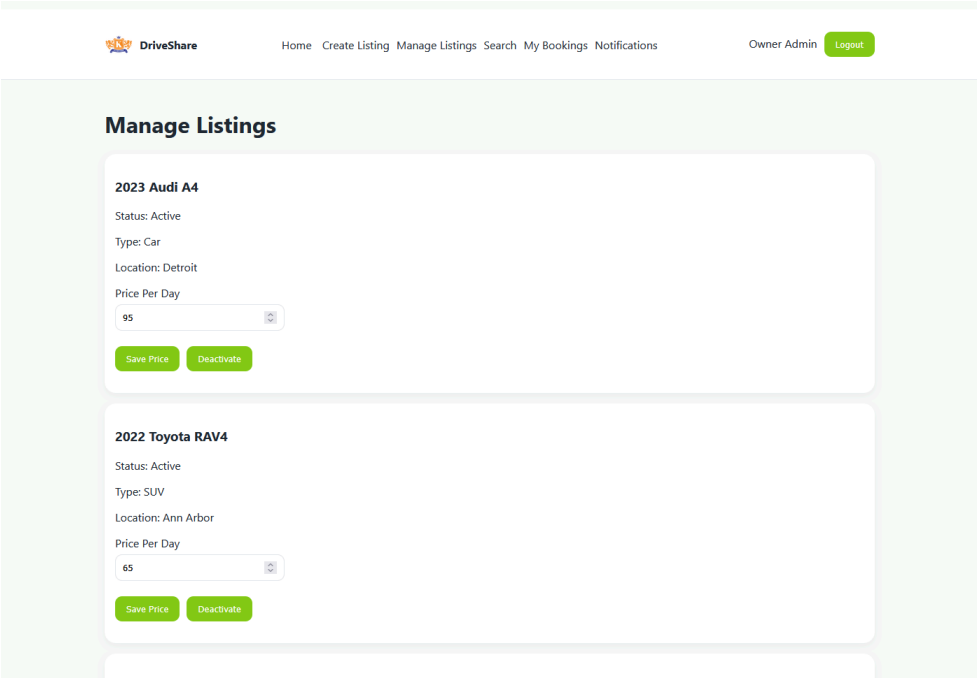
Login



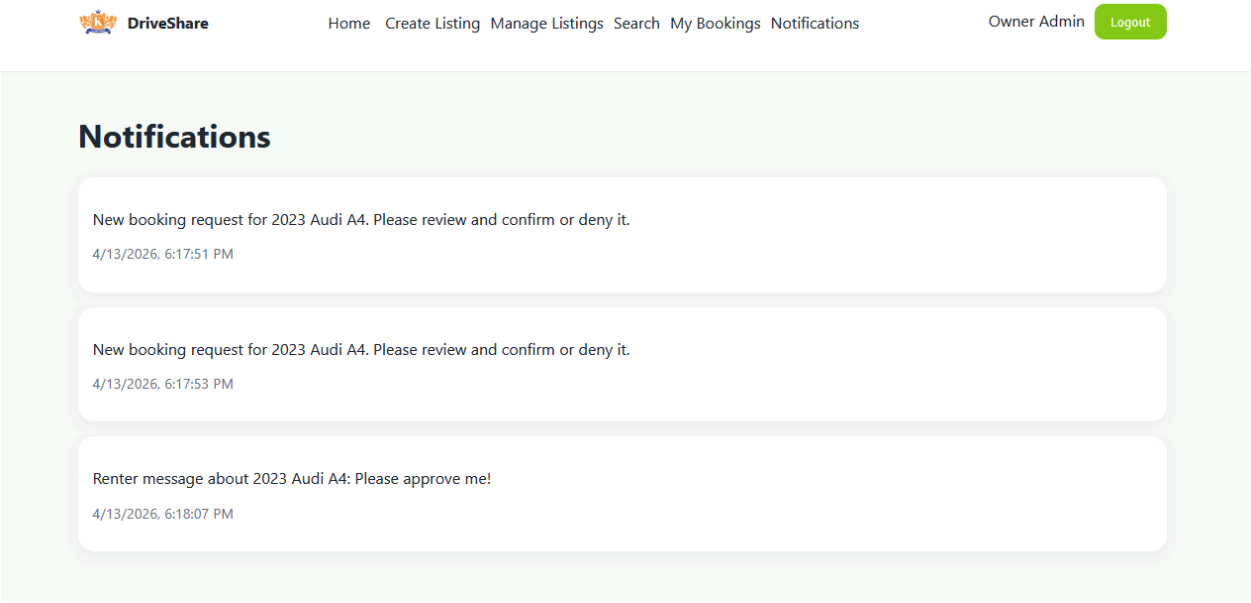
Create car listing (as owner)



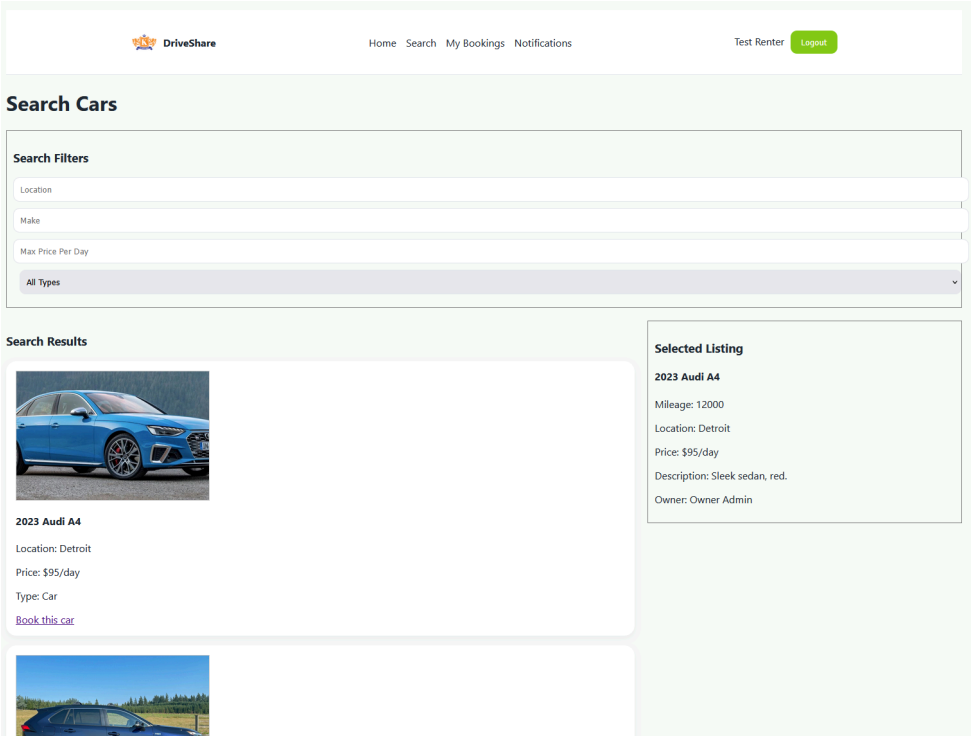
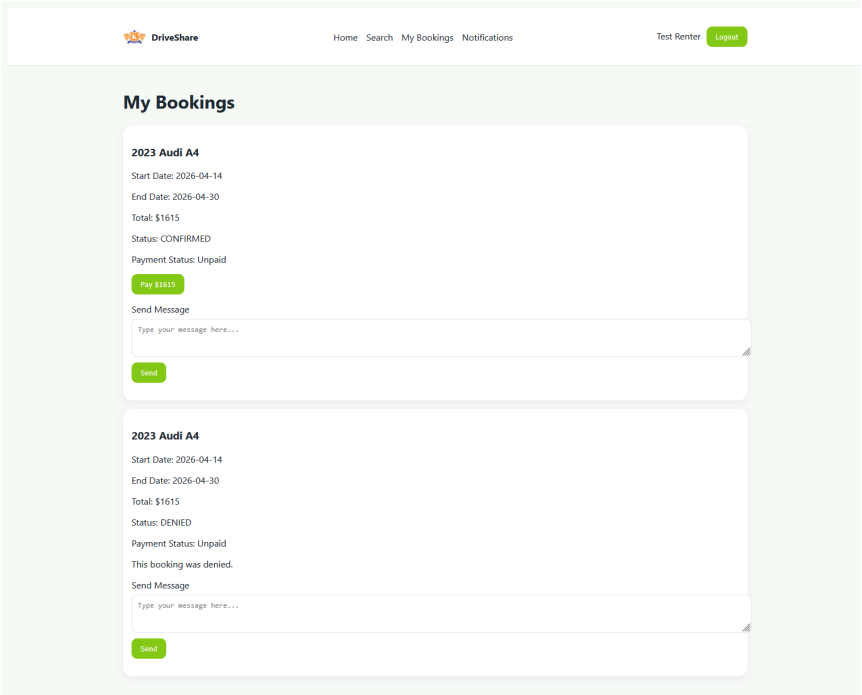
Manage car listings (as owner)



Notifications Page



Search Listings Page



My Bookings Page (renter)